

Modul 106 LB1

Teilnehmer: Arlind Sulejmani, Harun Siyad, Maximilian Kos, Mustafa Goat, Rilind Haradini

Kursleiter: Damien Vouillamoz

1. Aufgabe 1.1 Datenbankmodelle und Datenbanktheorie

1.1. Hierarchische Datenbank

- Hierarchische Speicherung der Daten in einem sequenziellen File (Land, Kanton, Ort/Ort, Bezirk, Gemeinde, Strasse, Hausnummer, Name, Vorname, Geburtsdatum)
- Neue Daten erfordern ein umkopieren, sortieren der Daten/File

1.2. Relationale und Objektrelationale Datenbanken

- Objekt(-relationale) Datenbanken erlauben benutzerdefinierte Datentypen und Objekte - Daten werden in Themenkreisen (Entitäten) in Form von Tabellen gespeichert
- Flexibler als Hierarchische Datenbanken
- Daten verschiedener Tabellen sind unabhängiger
- Einfach erweiterbar
- Unübersichtlicher und schwerfälliger

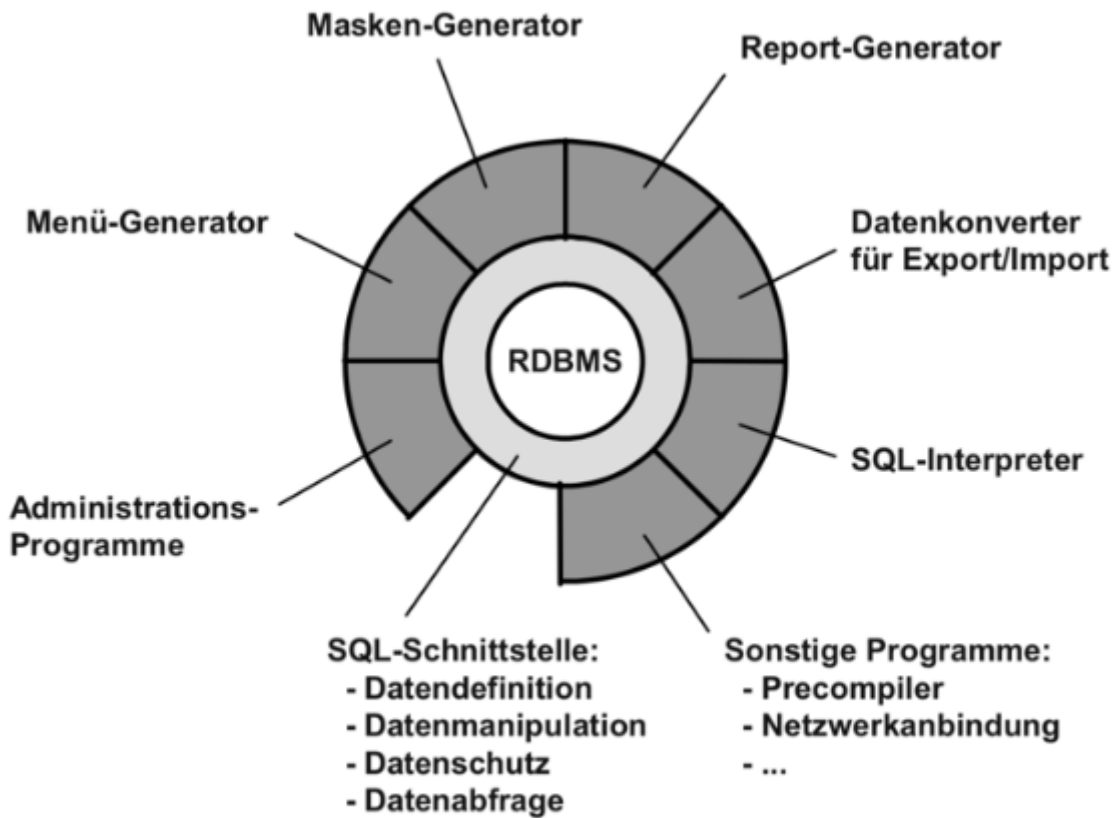
1.3. Objektorientierte Datenbank

- Objekt steht im Zentrum nicht die Tabelle
- Beinhaltet auf Methoden um Daten zu ändern
- Immer noch im Nischendasein

1.4. Datenbanktheorie zu relationalen Datenbanken

Eine relationale Datenbank:

- verwaltet Daten strukturiert und überschaubar
- regelt den Zugriff auf Datensätze
- bietet Schnittstellen zur flexiblen Interaktion durch externen Applikationen
- speichert Daten nicht redundant
- gewährleistet die Datenintegrität
- gewährleistet die Referenzielle Integrität



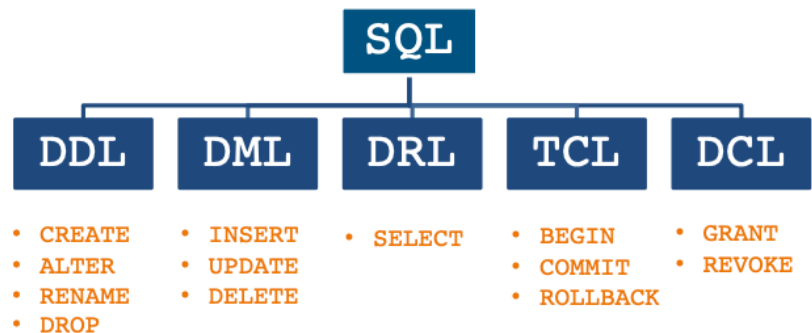
RDBMS = Relationales Datenbank-Management-System

1.5. Datenbanksprache SQL

Die Datenbanksprache besitzt vier Elemente:

1. Datendefinition (**DDL, Data Definition Language**)
2. Datenmanipulation (**DML, Data Manipulation Language**)
3. Datenabfrage (**DRL, Data Retrieval Language**)
4. Datenschutz (**DSL, Data Security Language**) —>
Besteht aus **TCL (Transaction Control Language)** und **DCL (Data Control Language)**

- **DDL**
DATA DEFINITION LANGUAGE
- **DML**
DATA MANIPULATION LANGUAGE
- **DRL**
DATA RETRIEVAL LANGUAGE
- **TCL**
TRANSACTION CONTROL LANGUAGE
- **DCL**
DATA CONTROL LANGUAGE



2. Aufgabe 1.2 Repetitionsfragen lösen

1. Was ist eine Relationale Datenbank?

- Eine relationale Datenbank ist eine Art von Datenbank, die Daten in Tabellen speichert, die über Schlüsselwörter und Verknüpfungen miteinander verbunden sind. Dies ermöglicht ein schnelleres und genaueres Abrufen von Informationen. Stellt referenzierte Integrität sicher. Datentypen müssen bei Einträgen eingehalten werden.

2. Nennen Sie drei verschiedene relationale Datenbank Management Systeme DBMS?

- MySQL
- Microsoft SQL Server
- Oracle Database

3. Welche Aufgaben erfüllt ein DBMS für Sie als Entwickler?

- Verwaltung und Speicherung von Daten in einer Datenbank
- Ermöglicht den Zugriff auf die Datenbank durch Abfragen und Aktionen mittels einer Abfragesprache
- Bietet Werkzeuge zur Verwaltung und Verwaltung der Datenbankstruktur, wie z.B. Erstellung, Änderung und Löschen von Tabellen.

4. Wie greifen Benutzer (keine Entwickler) auf eine Datenbank zu?

- Benutzer greifen auf eine Datenbank über eine Benutzeroberfläche (**GUI**) zu, die ihnen ermöglicht, Daten einzugeben, abzufragen, anzuzeigen und zu bearbeiten.
- Benutzer können auf die Datenbank über eine Anwendung (z.B. ein Programm oder eine Website) zugreifen, die speziell für den Zugriff auf die Datenbank entwickelt wurde.
- Benutzer können auf die Datenbank über Abfragesprachen (z.B. SQL) direkt zugreifen, wenn sie über die notwendigen Kenntnisse und Berechtigungen verfügen.

5. Handelt es sich bei SQL um eine Programmiersprache? (**NEIN**)

- SQL (Structured Query Language) ist eine spezielle Abfragesprache, die entwickelt wurde, um Daten in relationalen Datenbanken zu verwalten und abzufragen.
- SQL ist keine allgemeine Programmiersprache, sondern eine Domänen-spezifische Sprache, die ausschließlich für die Arbeit mit Datenbanken verwendet wird.
- SQL wird verwendet, um Aktionen wie Einfügen, Aktualisieren, Löschen und Abfragen von Daten in einer relationalen Datenbank durchzuführen.

6. Ist die Sprache SQL auf allen Datenbanken gleich?

- SQL ist eine standardisierte Abfragesprache, die von vielen Datenbankmanagementsystemen unterstützt wird.
- Allerdings gibt es Unterschiede in der Implementierung von SQL zwischen verschiedenen Datenbanken, insbesondere in Bezug auf erweiterte Funktionen und Erweiterungen.
- Einige Datenbanken haben ihre eigene Variante der SQL-Sprache, die speziell für die Verwendung mit dieser Datenbank optimiert wurde, wie z.B. T-SQL(Transact-SQL) von Microsoft SQL Server.

7. Was versteht man unter NoSQL Datenbank?

- NoSQL Datenbanken sind Datenbanken, die nicht auf die relationale Datenmodellierung und die Verwendung von SQL als Abfragesprache setzen.

- NoSQL Datenbanken verwenden andere Datenmodelle wie Dokument, Schlüssel-Wert, Graf oder Column-Family-basiertes Modell.
- NoSQL Datenbanken sind in der Lage, mit großen und unstrukturierten Datenmengen umzugehen und sind somit für Anwendungen mit hohen Anforderungen an Skalierbarkeit und Verfügbarkeit geeignet.

8. Nennen Sie drei NoSQL Datenbanksysteme?

- MongoDB
- Cassandra
- Redis

9. Was versteht man unter Datenbank Schema?

- Ein Datenbank-Schema beschreibt die Struktur und Organisation von Daten in einer Datenbank.
- Es enthält die Definitionen von Tabellen, Spalten, Indizes, Beziehungen und anderen Strukturelementen der Datenbank.
- Es legt die Regeln und Einschränkungen fest, die die Daten in der Datenbank einhalten müssen, um die Integrität der Daten zu gewährleisten.

10. Wozu dient ein Entity Relationship Model (ERM)?

- Ein Entity Relationship Model (ERM) dient dazu, die Beziehungen zwischen Entitäten in einer Datenbank zu beschreiben und darzustellen. Es hilft dabei, komplexe Datenstrukturen zu verstehen und zu analysieren.

11. Wie sieht ein Enhanced Entity Relationship (EER) Model aus?

- Ein Enhanced Entity Relationship (EER) Model ist ein Modell zur Visualisierung des Konzepts der Datenbankstruktur. Es besteht aus Entitäten, Beziehungen und Attributen, welche darstellen, wie die Entitäten miteinander in Beziehung stehen und wie die Attribute jeder Entität definiert sind.

12. Was bedeuten die Begriffe: Entität, Attribut und Tupel?

- **Entität:** Eine Entität ist ein Objekt, eine Person, ein Konzept oder ein Ereignis, das relevantes Wissen oder Daten besitzt und in einer Datenbank oder einem Datenmodell verarbeitet wird.
- **Attribut:** Ein Attribut ist ein Merkmal oder eine Eigenschaft einer Entität, das spezifische Informationen über die Entität enthält.
- **Tupel:** Ein Tupel ist eine Sammlung von Attributen, die zu einer Entität gehören. Es kann mehrere Attribute enthalten, die zusammenarbeiten, um eine einzelne Entität zu beschreiben.

13. Was versteht man unter Datenkonsistenz?

- Datenkonsistenz bezieht sich auf die Richtigkeit und Vollständigkeit von Daten in einer SQL-Datenbank.
- Datenkonsistenz bedeutet, dass die Daten in einer Datenbank korrekt und vollständig sind und dass sie sich nicht widersprechen.

14. Was bedeutet Redundanz im Zusammenhang mit einer Datenbank?

- Redundanz bedeutet, dass eine Datenbank mehr Daten als nötig speichert, um sicherzustellen, dass jede benötigte Information verfügbar ist und die Integrität der Daten gewährleistet ist.

15. Was versteht man unter Normalform?

- Die Normalform ist ein Kriterium für den Aufbau einer relationalen Datenbank. Es beinhaltet die Unterteilung von Tabellen in mehrere kleinere Tabellen, die jeweils ein spezifisches Merkmal enthalten, und die Verknüpfung dieser Tabellen mittels Fremdschlüssel.

16. Wie unterscheiden sich die erste, zweite und dritte Normalform?

- Die erste Normalform (1NF) besagt, dass alle Elemente in einer Tabelle einzelne Werte enthalten müssen und dass jede Spalte einen einzelnen Wert enthalten muss.
- Die zweite Normalform (2NF) erfordert, dass jedes Nicht-Schlüsselfeld in einer Tabelle vollständig durch den Primärschlüssel oder einen eindeutigen Kombinationsschlüssel determiniert wird.
- Die dritte Normalform (3NF) erfordert, dass jedes Nicht-Schlüsselfeld in einer Tabelle nur durch den Primärschlüssel determiniert wird und keine anderen Schlüsselfelder referenziert werden.

17. Was versteht man unter Primär- und Fremdschlüssel?

- Der Primärschlüssel ist ein eindeutiger Schlüssel, der jeder Zeile einer Tabelle zugeordnet ist.
- Der Fremdschlüssel ist ein Schlüssel, der eine Zeile in einer Tabelle mit einer Zeile in einer anderen Tabelle verknüpft.

18. Was ist referentielle Integrität?

- Referentielle Integrität ist ein Datenbanksystem, das sicherstellt, dass Daten nicht versehentlich oder absichtlich verändert oder gelöscht werden. Es schützt die Daten vor unerlaubten Änderungen, indem es Verbindungen zwischen Tabellen herstellt und Änderungen an den Daten validiert, bevor sie in der Datenbank gespeichert werden.

19. Was bringt Object-Relational Mapping (ORM)?

- ORM ermöglicht die Abbildung relationaler Datenbanken in objektorientierte Programmiersprachen. Es vereinfacht den Zugriff auf Datenbanken, da keine SQL-Befehle verwendet werden müssen, sondern die Datenbanken direkt über den Code angesprochen werden können.

20. Wie unterscheiden sich die Datentypen Char und Varchar?

- Der Datentyp Char speichert konstante Zeichenketten einer festen Länge, während der Datentyp Varchar Variable-Längen-Zeichenketten speichert.

21. Was ist ein Character Set und wie unterscheiden sich ASCII, LATIN1 und UTF-8?

- Ein Character Set ist eine Sammlung von Symbolen und Zeichen, die einem bestimmten Zweck dienen.
- ASCII (American Standard Code for Information Interchange) ist ein 7-bit-Code, der zur Darstellung von Texten in Englisch verwendet wird.

- LATIN1 (ISO-8859-1) ist ein 8-bit-Code, der zur Darstellung von Texten in Westeuropa verwendet wird.
- UTF-8 (Unicode Transformation Format 8-bit) ist ein 8-bit-Code, der für die Darstellung von Texten in verschiedenen Sprachen verwendet wird.

22. Wie unterscheiden sich die Datentypen Integer und Float?

- Integer-Variablen speichern ganzzahlige Werte, während Float-Variablen Gleitkommazahlen speichern.

23. Wozu verwendet man den Datentyp Decimal?

- Der Datentyp Decimal wird verwendet, um finanzielle Werte zu speichern.

24. Wie unterscheiden sich die Datentypen Timestamp und Datetime?

- Der Timestamp-Datentyp speichert eine einzelne Zeitangabe als Ganzzahl, während der Datetime-Datentyp speichert eine kombinierte Datum und Uhrzeitangabe als Zeichenfolge.

25. Wie unterscheiden sich 0 und NULL?

- 0 ist eine Zahl, die als Wert 0 dargestellt wird, während NULL als leerer Wert in Programmiersprachen verwendet wird. In der Datenbank wird NULL als fehlender Wert definiert, der keinen Wert hat.

3. Aufgabe 1.3 Erste Schritte mit MariaDB

- Hilfe anzeigen

```
mysql> \?
```

- Liste aller Datenbanken anzeigen

```
mysql> SHOW DATABASES;
```

- Mit einer bestimmten Datenbank verbinden

```
mysql> USE mysql;
```

- Alle Tabellen anzeigen

```
mysql> SHOW TABLES;
```

- Beschreibung (Struktur) einer Tabelle anzeigen

```
mysql> DESCRIBE user;  
mysql> DESC user;
```

4. Aufgabe 2.1 ERM des Datenbank Schemas entwerfen

SQL Script einfügen mit MariaDB mit

SOURCE C:/Users/User/Documents/script.sql

```
DROP DATABASE IF EXISTS `pizzashop`;  
CREATE DATABASE `pizzashop`;  
USE `pizzashop`;  
  
CREATE TABLE `customer`(  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `firstname` VARCHAR(45) NOT NULL,  
  `lastname` VARCHAR(45) NOT NULL,  
  `postcode` INT NOT NULL,  
  `location` VARCHAR(45) NOT NULL,  
  `email` VARCHAR(255) NOT NULL UNIQUE,  
  `phone_number` VARCHAR(255) NOT NULL UNIQUE,  
  PRIMARY KEY (`id`)  
);  
  
CREATE TABLE `order`(  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `amount` INT NOT NULL,  
  `order_date` DATETIME NOT NULL DEFAULT NOW(),  
  `delivery_date` DATETIME,  
  `fk_customer_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  FOREIGN KEY (`fk_customer_id`) REFERENCES `customer`(`id`)  
);  
  
CREATE TABLE `product_category`(  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`id`)  
);  
  
CREATE TABLE `product`(  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `description` TEXT NOT NULL,  
  `price` DECIMAL(5,2),  
  `fk_product_category_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  FOREIGN KEY (`fk_product_category_id`) REFERENCES `product_category`(`id`)  
);  
  
CREATE TABLE `order_has_product` (  
  `fk_order_id` INT NOT NULL,
```

```
`fk_product_id` INT NOT NULL,  
FOREIGN KEY (`fk_order_id`) REFERENCES `order`(`id`),  
FOREIGN KEY (`fk_product_id`) REFERENCES `product`(`id`)  
);
```

5. Aufgabe 2.4 Schema mit SQL/DDL bearbeiten

1. Sie wollen in der Kundentabelle auch die mobile Telefonnummer speichern. Fügen Sie eine entsprechende Spalte hinzu

```
ALTER TABLE `customer` ADD `mobile` VARCHAR(255);
```

2. Ändern Sie den Namen der Spalte für die Produktbezeichnung

```
ALTER TABLE `product` CHANGE `name` `product_name` VARCHAR(255) NOT NULL;  
ALTER TABLE `product` CHANGE `product_name` `name` VARCHAR(255) NOT NULL;
```

3. Ändern Sie den Datentyp des Produktpreises auf DECIMAL(6,2) UNSIGNED

```
ALTER TABLE `product` MODIFY `price` DECIMAL(6,2) UNSIGNED;
```

4. Setzen Sie nachträglich NOT NULL für den Produktpreis

```
ALTER TABLE `product` MODIFY `price` DECIMAL(6,2) NOT NULL;
```

5. Fügen Sie ein neues Attribut (created_at, DATETIME) in die Produkttabelle ein und stellen Sie sicher, dass dieses Feld automatisch mit dem aktuellen Zeitpunkt bei einem INSERT befüllt wird.

```
ALTER TABLE `product` ADD `created_at` DATETIME NOT NULL DEFAULT NOW();
```

6. Entfernen Sie die Spalte für die mobile Telefonnummer wieder

```
ALTER TABLE `customer` DROP `mobile`;
```

7. Entfernen Sie den Foreign Key Constraint vom Postleitzahlen Fremdschlüssel aus der Kundentabelle

```
SHOW CREATE TABLE `customer`; -- Name des CONSTRAINTs herausfinden (z.B.  
customer_ibfk_1)
```

```
ALTER TABLE `customer` DROP CONSTRAINT `customer_ibfk_1`;
```

8. Fügen Sie den Foreign Key Constraint wieder hinzu

```
ALTER TABLE `customer` ADD foreign key(`fk_zip_id`) REFERENCES `zip`(`id`);
```

6. Lernziele LB1

- Ich kenne die verschiedenen Arten von Datenbanken und deren hauptsächlichen Unterschiede
 - Relationale Datenbanken: Speichern Daten in Tabellen mit Beziehungen zwischen ihnen.
 - NoSQL-Datenbanken: Speichern Daten in Dokumenten, die keine Beziehungen zu anderen Dokumenten haben.
 - Objektorientierte Datenbanken: Speichern Daten in Objekten, die miteinander verknüpft sind.
 - Grafische Datenbanken: Speichern Daten in einem Netzwerk von Knoten und Kanten.
- Ich kenne die Aufgaben eines RDBMS sowie den Aufbau eines Datenbanksystems
 - **RDBMS:** Ein Relationales Datenbankmanagementsystem (RDBMS) ist ein Programm, mit dem Daten in einer relationalen Datenbank organisiert und gespeichert werden können. Das RDBMS ermöglicht es Benutzern, Daten zu erfassen, abzurufen, zu bearbeiten und zu ändern.
 - **Aufbau eines Datenbanksystems:** Ein Datenbanksystem besteht aus einer relationalen Datenbank, einer Datenbankmanagementsystem-Software (RDBMS) und einer Benutzer-Schnittstelle. Die relationale Datenbank ist eine Sammlung von Tabellen, in denen Daten gespeichert werden. Die RDBMS-Software verwaltet die Datenbank und stellt verschiedene Funktionen zur Verfügung, mit denen Benutzer auf die Daten zugreifen und sie verwalten können. Die Ben
- Ich kenne die verschiedenen Elemente der strukturierten Abfragesprache SQL
 - **SELECT** - Der SELECT-Befehl wird verwendet, um Daten aus einer Datenbank abzurufen.
 - **FROM** - Der FROM-Befehl wird verwendet, um eine Tabelle oder mehrere Tabellen auszuwählen, aus denen Daten abgerufen werden sollen.
 - **WHERE** - Der WHERE-Befehl wird verwendet, um bestimmte Kriterien für die Datenabfrage auszuwählen.
 - **ORDER BY** - Der ORDER BY-Befehl wird verwendet, um die Ergebnisse einer Abfrage zu sortieren.
 - **GROUP BY** - Der GROUP BY-Befehl wird verwendet, um die Ergebnisse einer Abfrage zu gruppieren.
 - **HAVING** - Der HAVING-Befehl wird verwendet, um eine Bedingung auf die abgerufenen Gruppen anzuwenden.
 - **UPDATE** - Der UPDATE-Befehl wird verwendet, um Daten in einer Datenbank zu aktualisieren.
 - **INSERT** - Der INSERT-Befehl wird verwendet, um Daten in einer Datenbank einzufügen.

- **DELETE** - Der DELETE-Befehl wird verwendet, um Daten aus einer Datenbank zu löschen.
- Ich kann MariaDB installieren
- Ich kann die Datenbank-Konsole öffnen und SQL Statements eingeben
- Ich kann alle Datenbanken im System anzeigen
 - **SHOW DATABASES**
- Ich kann mich mit einer Datenbank verbinden
- Ich kann die Datenbank Struktur anzeigen
 - **SHOW TABLES**
- Ich kann ein Datenbank Schema in der 3. Normalform entwerfen
- Ich kenne dies SQL Befehle die zur Data Definition Language (DDL) gehören
 - **CREATE**: Erstellung einer neuen Tabelle, View, Index, etc.
 - **ALTER**: Ändern einer vorhandenen Tabelle, View, Index, etc.
 - **DROP**: Löschen einer vorhandenen Tabelle, View, Index, etc.
 - **TRUNCATE**: Löschen aller Datensätze aus einer Tabelle.
- Ich kann Datenbanken und Tabellen mit SQL erzeugen
 - **CREATE DATABASE** `dbname;`
- Ich kann Datenbanken und Tabellen mit SQL löschen
 - **DELETE FROM** `table_name WHERE condition;`
- Ich kann die Tabellenstruktur mit SQL ändern
 - Mit CREATE, ALTER und DROP können Änderungen an Tabellenstrukturen vorgenommen werden. CREATE ermöglicht das Erstellen einer neuen Tabelle, ALTER erlaubt es, bestehende Tabellen zu bearbeiten und DROP kann eine Tabelle löschen.

Modul 106 LB2

7. Aufgabe 3.1 Datensätze erfassen

1. Erfassen Sie mindestens 3 sinnvolle Produktkategorien

```
INSERT INTO `product_category` (`name`) VALUES
("Pizza Margherita"),
("Schinken"),
("Vegan");
```

2. Erfassen Sie mindestens 5 verschiedene Produkte

```
INSERT INTO `product` (`id`, `name`, `description`, `price`,  
`fk_product_category_id`) VALUES  
(1, 'Smartphone', 500, 1, 1),  
(2, 'Laptop', 1500, 2, 2),  
(3, 'Tablet', 800, 1, 3),  
(4, 'TV', 1000, 3, 2),  
(5, 'Printer', 200, 4, 1);
```

3. Erfassen Sie mindestens 5 vollständige Kunden

```
INSERT INTO `customer` (`firstname`, `lastname`, `postcode`, `location`, `email`,  
`phone_number`) VALUES  
( 'Arlind', 'Sulejmani', '8055', 'Zuerich', 'sulejmaniarlind5@gmail.com', '+41 76  
222 22 22'),  
( 'Harun', 'Siyad', '8050', 'Zuerich', 'harunsiyad@gmail.com', '+41 76 333 33 33'),  
( 'Maximilian', 'Kos', '8912', 'Obfelnden', 'maxkos@gmail.com', '+41 76 444 44  
44'),  
( 'Maxi', 'Kos', '8912', 'Obfelnden', 'maxikos@gmail.com', '+41 76 555 55 55'),  
( 'Max', 'Kos', '8912', 'Obfelnden', 'maxos@gmail.com', '+41 76 666 666 66');
```

4. Erfassen Sie mindestens 4 vollständige Bestellungen, davon mind 3 die bereits ausgeliefert wurden.

```
INSERT INTO `order` (`amount`, `order_date`, `delivery_date`, `fk_customer_id`) VALUES  
(1, '2022-01-20 19:00:00', '2022-01-20 21:00:00', 1),  
(2, '2022-01-20 09:00:00', '2022-01-20 12:00:00', 2),  
(3, '2022-01-20 01:00:00', '2022-01-20 06:00:00', 3),  
(4, '2022-01-20 05:00:00', NULL, 4);
```

5. Fassen Sie alle INSERT-Statements in einer Transaktion zusammen

```
BEGIN;  
  
INSERT INTO `customer` (`firstname`, `lastname`, `postcode`, `location`, `email`,  
`phone_number`) VALUES  
( 'Arlind', 'Sulejmani', '8055', 'Zuerich', 'sulejmaniarlind5@gmail.com', '+41 76  
222 22 22'),  
( 'Harun', 'Siyad', '8050', 'Zuerich', 'harunsiyad@gmail.com', '+41 76 333 33 33'),  
( 'Maximilian', 'Kos', '8912', 'Obfelnden', 'maxkos@gmail.com', '+41 76 444 44  
44'),  
( 'Maxi', 'Kos', '8912', 'Obfelnden', 'maxikos@gmail.com', '+41 76 555 55 55'),  
( 'Max', 'Kos', '8912', 'Obfelnden', 'maxos@gmail.com', '+41 76 666 666 66');  
  
INSERT INTO `product_category` (`name`) VALUES  
( "Pizza Margherita"),  
( "Schinken"),
```

```
("Vegan");

INSERT INTO `product` (`id`, `description`, `price`, `fk_product_category_id`)
VALUES
(1, 'Smartphone', 500, 1),
(2, 'Laptop', 1500, 2),
(3, 'Tablet', 800, 3),
(4, 'TV', 1000, 1),
(5, 'Printer', 200, 2);

INSERT INTO `order` (`amount`, `order_date`, `delivery_date`, `fk_customer_id`)
VALUES
(1, '2022-01-20 19:00:00', '2022-01-20 21:00:00', 1),
(2, '2022-01-20 09:00:00', '2022-01-20 12:00:00', 2),
(3, '2022-01-20 01:00:00', '2022-01-20 06:00:00', 3),
(4, '2022-01-20 05:00:00', NULL, 4);
```

6. Stellen Sie sicher, dass Sie mindestens eine Produktkategorie haben, die kein Produkt besitzt

7. Stellen Sie sicher, dass Sie mindestens ein Produkt haben, dass nie bestellt wurde

8. Benutzer erstellen

```
CREATE USER backup@'localhost' IDENTIFIED BY '123';
SET PASSWORD FOR backup@'localhost' = PASSWORD('B4ckU9u3er');
GRANT SELECT ON pizzashop.* to backup@'localhost';

CREATE USER backoffice@'%' IDENTIFIED BY '123';
GRANT SELECT ON pizzashop.* to backoffice@'%';
GRANT SELECT, UPDATE, INSERT, DELETE ON pizzashop.customer to backoffice@'%';
GRANT SELECT, UPDATE, INSERT, DELETE ON pizzashop.product to backoffice@'%';
GRANT SELECT, UPDATE, INSERT, DELETE ON pizzashop.product_category to
backoffice@'%';

CREATE USER sales@'%' IDENTIFIED BY '123';
SET PASSWORD FOR sales@'%' = PASSWORD('B4ckU9u3er');
GRANT SELECT ON pizzashop.customer to sales@'%';
GRANT SELECT ON pizzashop.product_category to sales@'%';
GRANT SELECT ON pizzashop.order_has_product to sales@'%';
GRANT SELECT, UPDATE, INSERT, DELETE ON pizzashop.`order` to sales@'%';
GRANT SELECT, UPDATE, INSERT, DELETE ON pizzashop.product_category to sales@'%';
```

9. Aufgabe 4.1: Einfache Datenabfragen

1. Liste aller Produkte

```
SELECT description FROM product;
```

2. Liste aller Kategorien

```
SELECT name FROM product_category;
```

3. Liste aller Kunden. Geben Sie nur Vorname, Nachname und Emailadresse aus

```
SELECT firstname, lastname, email FROM customer;
```

4. Liste aller Bestellungen sortiert nach Bestelldatum

```
SELECT * FROM `order` ORDER BY `order_date` ASC;
```

5. Liste aller Produkte absteigend sortiert nach Preis

```
SELECT * FROM `product` ORDER BY `price` DESC;
```

6. Liste der teuersten 3 Produkte

```
SELECT price FROM product ORDER BY price DESC LIMIT 3
```

7. Liste der günstigsten 3 Produkte

```
SELECT price FROM product ORDER BY price ASC LIMIT 3
```

10. Aufgabe 4.2: Funktionen anwenden

1. Berechnen Sie die Quadratwurzel aller Produktpreise.

```
SELECT SQRT(price) FROM product;
```

2. Geben Sie den Namen des Monats aus dem Datum der Bestellungen aus.

```
SELECT MONTH(ordered_at) FROM order_entry;
```

3. Zählen Sie die Anzahl Buchstaben in den Vornamen der Kunden.

```
SELECT LENGTH(firstname) FROM customer;
```

4. Liste der Email Adressen aller Kunden. Teilen Sie die Adresse in zwei Spalten auf Account und Domain.
z.B. hans@muster.com: hans, muster.com

```
SELECT SUBSTRING_INDEX(email, '@', 1) AS account,  
SUBSTRING_INDEX(email, '@', -1) AS domain  
FROM customer;
```

5. Geben Sie die Initialen der Kunden in einer Spalte aus. z.B. Hans Muster: HM

```
SELECT CONCAT(LEFT(firstname, 1), LEFT(lastname, 1)) AS Initialen  
FROM customer;
```

6. Berechnen Sie die 8% Mehrwertsteuer, die in den Preisen inbegriffen ist (Optional: Runden Sie die MwSt auf 5 Rappen)

```
SELECT ROUND(price * 0.08, 2) FROM product;
```

7. Geben Sie die Anzahl Datensätze ihrer Produkttabelle aus.

```
SELECT COUNT(*) FROM product;
```

8. Berechnen Sie Mindest-, Höchst- und Durchschnittspreis aller Produkte

```
SELECT MIN(price), MAX(price), AVG(price) FROM product;
```

11. Aufgabe 4.3: WHERE Bedingungen

1. Produkt mit dem Primärschlüssel 5

```
SELECT * FROM product WHERE id = 5;
```

2. Kunden deren Primärschlüssel kleiner ist als 3

```
SELECT * FROM customer WHERE id < 3;
```

3. Kunden deren Primärschlüssel kleiner ist als 3 oder grösser als 8

```
SELECT * FROM customer WHERE id < 3 OR id > 8;
```

4. Bestellungen mit Primärschlüssel zwischen 3 und 7

```
SELECT * FROM order_entry WHERE id BETWEEN 3 AND 7;
```

5. Kunden mit den Primärschlüsseln 1,3,5 und 6

```
SELECT * FROM customer WHERE id IN (1, 3, 5, 6);
```

6. Bestellungen deren Lieferdatum NULL ist

```
SELECT * FROM order_entry WHERE delivered_at IS NULL;
```

7. Produkte die mehr kosten als der Durchschnitt *Hinweis: Berechnen Sie zuerst den Durchschnittspreis mit der Funktion AVG()*

```
SELECT * FROM product WHERE price > (SELECT AVG(price) FROM product);
```

12. Aufgabe 4.4: LIKE Patterns

1. Kunden deren Vorname mit "f" beginnt

```
SELECT * FROM `customer` WHERE `firstname` LIKE 'f%';
```

2. Kunden deren Nachname mit dem Buchstaben "r" endet

```
SELECT * FROM `customer` WHERE `lastname` LIKE '%r';
```

3. Kunden deren Nachname ein "e" enthält

```
SELECT * FROM `customer` WHERE `lastname` LIKE "%e%";
```

4. Kunden deren Vorname aus 5 Buchstaben besteht

```
SELECT * FROM `customer` WHERE `firstname` LIKE "_____";
```

5. Kunden deren Nachname an der zweitletzter Stelle ein "e" haben

```
SELECT * FROM `customer` WHERE `lastname` LIKE "%e_";
```

13. Aufgabe 5.1: Datenbank analysieren

1. Welche Tabellen sind in der Datenbank vorhanden und wie sind diese mit Primär- und Fremdschlüssel verbunden?
 - actor - Mit dem Primärschlüssel actor_id verbunden.
 - address - Mit dem Primärschlüssel address_id verbunden.
 - category - Mit dem Primärschlüssel category_id verbunden.
 - city - Mit dem Primärschlüssel city_id verbunden.
 - country - Mit dem Primärschlüssel country_id verbunden.
 - customer - Mit dem Primärschlüssel customer_id verbunden.
 - film - Mit dem Primärschlüssel film_id verbunden.
 - film_actor - Mit den Fremdschlüsseln film_id und actor_id verbunden.
 - film_category - Mit den Fremdschlüsseln film_id und category_id verbunden.
 - film_text - Mit dem Primärschlüssel film_id verbunden.
 - inventory - Mit dem Primärschlüssel
2. Was wird in dieser Datenbank repr. in den einzelnen Tabellen gespeichert?
 - Recht selbstverständlich sobald man die Tabellen Namen liest.
3. Was ist der Umfang der in der Datenbank gespeicherten Daten?
 - Die Sakila-Datenbank ist eine Beispiel-Datenbank, die von MySQL entwickelt wurde. Sie enthält Daten zu Filmen, Kunden, Mieten und anderen Aspekten eines fiktiven Videostores.
4. Welchen Geschäftsprozess bildet die Datenbank Sakila ab?
 - Die Datenbank Sakila bildet ein Geschäftsprozess des Verleihs von Filmen ab. Sie enthält Datensätze für Kunden, Filme, Verleihe, Mietgebühren, Zahlungen und andere. Mit Sakila können Kunden Filme ausleihen, verfolgen, wie lange sie den Film ausgeliehen haben, wann sie ihn zurückgeben müssen, wie viel sie bezahlen müssen, ob sie ihren Verleih aktualisieren müssen, und mehr.

14. Aufgabe 5.2: SQL Abfragen machen

1. Anzahl Datensätze in der Tabelle payment

```
SELECT COUNT(*) FROM `payment`;
```

2. Anzahl Schauspieler mit dem Namen Julia

```
SELECT * FROM `actor` WHERE `first_name` LIKE 'Julia';
```

3. Anzahl inaktive Kunden

```
SELECT COUNT(*) FROM `customer` WHERE `active` LIKE "0";
```

4. Durchschnittliche Länge der Filme mit Rating "PG"

```
SELECT AVG(length) FROM `film` WHERE `rating` = 'PG';
```

5. Ausleihen, die noch nicht zurück gebracht wurden, sortiert nach Ausleihdatum (Format: dd.mm.YYYY)

```
SELECT * FROM `rental` WHERE `return_date` IS NULL ORDER BY `rental_date`  
ASC;
```

6. Berechnen Sie die durchschnittliche Ausleihdauer in Tagen

```
SELECT AVG(DATEDIFF(`return_date`, `rental_date`)) AS avg_rental_days FROM  
rental;
```

7. Liste der Vornamen von Schauspielern, deren Vorname nur 3 Buchstaben lang ist. Zeigen Sie keine doppelten Vornamen an

```
SELECT DISTINCT `first_name` FROM `actor` WHERE LENGTH(`first_name`)=3;
```

8. Erstellen Sie eine Liste der Schauspieler, deren Nachname mit 'B' beginnt und an zweitletzter Stelle ein 'e' haben. Zeigen Sie keine doppelten Namen an

```
SELECT DISTINCT `first_name`, `last_name` FROM `actor` WHERE `first_name`  
LIKE 'B%' AND `last_name` LIKE '_e%';
```

9. Zählen Sie bei allen Datensätzen der Tabelle "rental" 12 Jahre zu Ausleih- und Rückgabedatum dazu

```
SELECT COUNT(*)FROM `rental` WHERE DATE_ADD(`rental_date`, INTERVAL 12 YEAR)  
> `return_date`;
```

10. Erstellen Sie eine neue Kategorie namens "Art"

```
INSERT INTO `category` (name) VALUES ('Art');
```

15. Aufgabe 5.3: Indizes erstellen und testen

```
CREATE DATABASE INDEX_TEST;
USE INDEX_TEST;
CREATE TABLE babyname (
  year INT(4),
  name VARCHAR(11),
  percent FLOAT,
  gender VARCHAR(5));

-- kopiere baby-names.csv zu C:\Program Files\MariaDB 10.6\data\baby-names.csv
LOAD DATA INFILE './baby-names.csv' INTO TABLE babyname FIELDS TERMINATED BY ',';

-- SELECT * FROM babyname;
-- SELECT * FROM babyname WHERE name = 'Markus';

-- MariaDB [INDEX_TEST]> EXPLAIN SELECT * FROM babyname WHERE name = 'Markus';
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
-- | id   | select_type | table   | type | possible_keys | key   | key_len | ref
| rows  | Extra      |
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
-- |    1 | SIMPLE      | babyname | ALL  | NULL          | NULL  | NULL    | NULL
| 516000 | Using where |
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+

-- jetzt schneller

CREATE INDEX index_name ON babyname(name);
SELECT * FROM babyname WHERE name = 'Markus';

-- MariaDB [INDEX_TEST]> EXPLAIN SELECT * FROM babyname WHERE name = 'Markus';
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
-- | id   | select_type | table   | type | possible_keys | key           | key_len |
ref  | rows  | Extra      |
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
-- |    1 | SIMPLE      | babyname | ref  | index_name    | index_name    | 36      |
const | 94    | Using index condition |
-- +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+

-- Ohne index: 94 rows in set (0.186 sec)
-- mit index: 94 rows in set (0.002 sec)
```

16. Aufgabe 6.1: CSV Dateien exportieren

1. Erstellen Sie eine Liste aller Kunden und speichern Sie sie als CSV Datei ab. Folgende Felder müssen exportiert werden:

1. `customer.customer_id`
2. `customer.first_name`
3. `customer.last_name`
4. `customer.email`
5. `customer.create_at` (Format DD.MM.YYYY)

```
-- use sakila;  
SELECT customer.customer_id, customer.first_name, customer.last_name,  
customer.email, DATE_FORMAT(customer.create_date, '%d.%m.%Y') AS create_at  
FROM customer  
INTO OUTFILE 'C:/Program Files/MariaDB 10.6/data/sakila.csv'  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n';
```

2. Öffnen Sie die Adressliste in Excel.

- ☒ wow

3. Wiederholen Sie den Vorgang. Nehmen Sie diesmal den ; als Feldtrenner und " als Feld-Enclosing.

```
SELECT customer.customer_id, customer.first_name, customer.last_name,  
customer.email, DATE_FORMAT(customer.create_date, '%d.%m.%Y') AS create_at  
FROM customer  
INTO OUTFILE 'C:/Program Files/MariaDB 10.6/data/sakila2.csv'  
FIELDS TERMINATED BY ';'   
ENCLOSED BY '"'   
LINES TERMINATED BY '\n';
```

17. Aufgabe 6.2: SQL-Dumps erzeugen und einlesen

1. Erstellen Sie einen SQL-Dump ihrer Sakila Datenbank.

```
mysqldump.exe -u root -p sakila > c:/users/user/desktop/dumps/dump-  
sakila.sql
```

2. Laden Sie den SQL-Dump in eine neue Datenbank namens sakila_copy.

```
CREATE DATABASE sakila_copy;  
exit
```

```
mysql.exe -u root -p sakila_copy < c:/users/user/desktop/dumps/dump-sakila.sql
```

3. Erstellen Sie einen SQL-Dump aller Datenbanken auf Ihrem Server als Backup.

```
mysqldump.exe -u root -p --all-databases > c:/users/user/desktop/dumps/dump-all-databases.sql
```

18. Aufgabe 6.3: Schweizer Postleitzahlen importieren

1. Überfliegen Sie das Factsheet post-match-zip-factsheet.pdf.
 - ☒ wow
2. Schauen Sie sich den Inhalt der 4 Datenfiles an.
 - ☒ wow
3. Studieren Sie das Datenfile PLZ Light / plz_l_20130901.zip sowie deren Referenz im post-match-zip-factsheet.pdf im Detail
 - ☒ wow
4. Erstellen Sie eine neue Datenbank und realisieren eine Tabelle für das Datenfile gemäss Factsheet.

```
CREATE DATABASE post;
```

```
CREATE TABLE und so wart han grad kei bock
```

5. Laden Sie den Inhalt des Datenfiles in die Tabelle.

6. Kontrollieren Sie, ob alle Datensätze vorhanden sind.

- ☒ wow

19. Update sql example

```
UPDATE Customers  
SET ContactName = 'Juan', City = 'Madrid'  
WHERE CustomerID = 1;
```

20. delete sql example

```
DELETE FROM Customers  
WHERE CustomerID = 1;
```

20. Aufgabe 7.1: Datensätze Gruppieren

1. Durchschnittspreis alle Produkte *Ohne GROUP BY*

```
select AVG(price) from product;
```

2. Anzahl Produkte pro Produktkategorie *Hinweis: verwenden Sie den Fremdschlüssel für die Gruppierung*

```
SELECT COUNT(*) AS `Anzahl Kategorien` FROM product GROUP BY fk_category_id;
```

3. Anzahl Kunden pro Postleitzahl-ID

```
SELECT COUNT(*) AS `KUNDEN PRO POSTLEITZAHL` FROM customer GROUP BY id;
```

4. Höchst-, Mindest- und Durchschnittspreis aller Produktkategorien

```
select AVG(price), MIN(price), MAX(price) from product GROUP BY price;
```

5. Durchschnittspreis der Produktkategorie mit Fremdschlüssel 2

```
SELECT AVG(price) FROM product WHERE fk_category_id = 2;
```

6. Produktkategorien mit einem Durchschnittspreis grösser als das Resultat von Aufgabe 1

```
SELECT price FROM product WHERE price > 13.1;
```

21. Aufgabe 7.2: Tabellen verbinden

1. Adressliste aller Kunden mit Postleitzahl und Ort

```
SELECT customer.firstname, customer.lastname, customer.address, zip.zip,  
zip.city  
FROM customer  
JOIN zip ON customer.fk_zip_id = zip.id;
```

2. Alle Produkte mit Beschreibung und Name der Kategorie

```
SELECT product.name, product.description, category.category AS category_name
FROM product
JOIN category ON product.fk_category_id = category.id;
```

3. Alle Bestellungen mit Adresse, Postleitzahl, Bestell- und Lieferdatum

```
SELECT customer.address, zip.zip, order_entry.ordered_at,
order_entry.delivered_at
FROM customer
JOIN order_entry ON customer.id = order_entry.fk_customer_id
JOIN zip ON zip.id = customer.fk_zip_id;
```

4. Alle Bestellungen mit Email des Kunden, Bestelldatum, Produktbezeichnung, Anzahl, Preis und Summe

Hinweis: Die Summe einer Bestellung müssen Sie aus Anzahl und Preis berechnen

```
SELECT customer.email, order_entry.ordered_at, product.name,
product_order_entry.amount, product_order_entry.price,
product_order_entry.amount * product_order_entry.price AS `sum`
FROM customer
INNER JOIN order_entry ON order_entry.fk_customer_id = customer.id
INNER JOIN product_order_entry ON product_order_entry.fk_order_entry_id =
order_entry.id
INNER JOIN product ON product_order_entry.fk_product_id = product.id;
```

5. Kategorienamen mit Anzahl darin enthaltener Produkte

```
SELECT category.category AS category, COUNT(product.id) AS product_count
FROM category
INNER JOIN product ON category.id = product.fk_category_id
GROUP BY category.id;
```

6. Kundennamen mit Anzahl Bestellungen

```
SELECT customer.firstname, customer.lastname,
COUNT(order_entry.fk_customer_id) AS order_count
FROM customer
JOIN order_entry ON customer.id = order_entry.fk_customer_id
GROUP BY customer.firstname, customer.lastname;
```

7. Erhöhen Sie den Preis aller Produkte, die einer bestimmten Kategorie zugeordnet sind. Verwenden Sie einen Join im Update Statement.

```
UPDATE product
INNER JOIN category ON product.id = category.id
SET product.price = product.price + (product.price * 0.1)
WHERE category.category = 'Dessert';
```

8. Löschen Sie alle Bestellungen, die von Kunden an einer bestimmten Postleitzahl getätigt wurden. Verwenden Sie einen Join im Delete Statement.

```
DELETE order_entry
FROM order_entry
JOIN customer ON order_entry.fk_customer_id = customer.id
JOIN zip ON customer.fk_zip_id = zip.id WHERE zip.zip = 8049;
```

22. Aufgabe 7.3: Outer Joins

1. Liste aller Produkte mit Produktkategorien *Hinweis: Zeigen Sie auch die Kategorien an, die keine Produkte haben.*

```
SELECT product.name, category.category
FROM product
LEFT JOIN category ON product.id = category.id;
```

2. Liste aller Produktkategorien und Anzahl darin enthaltener Produkte. Zeigen Sie alle Kategorien an und verwenden Sie COUNT().

```
SELECT category.category, COUNT(product.id) AS product_count
FROM category
LEFT JOIN product
ON category.id = product.fk_category_id
GROUP BY category.category;
```

3. Liste der Namen aller Kunden und Anzahl Bestellungen aller Kunden.

```
SELECT customer.firstname, customer.lastname,
COUNT(order_entry.fk_customer_id) AS order_count
FROM customer
LEFT JOIN order_entry
ON customer.id = order_entry.fk_customer_id
GROUP BY customer.firstname, customer.lastname;
```

4. Liste der Produktkategorien, die keine Produkte enthalten

```
SELECT category.category
FROM category
LEFT JOIN product
ON category.id = product.fk_category_id
WHERE product.id IS NULL;
```

5. Liste der Produkte, die noch nie bestellt wurden

```
SELECT product.name
FROM product
LEFT JOIN product_order_entry
ON product.id = product_order_entry.fk_product_id
WHERE product_order_entry.id IS NULL;
```

cooles Beispiel

```
SELECT tmp.nummer FROM (SELECT COUNT(*) AS nummer From actor) as tmp;
```

23. Aufgabe 7.4: Subqueries

1. Produkte die mehr kosten als der Durchschnitt

```
SELECT * FROM product WHERE price > (SELECT AVG(price) FROM product);
```

2. Produkte die weniger kosten als der Durchschnitt

```
SELECT * FROM product WHERE price < (SELECT AVG(price) FROM product);
```

3. Bezeichnung und Preis des teuersten Produktes

```
SELECT name, price FROM product ORDER BY price DESC LIMIT 1;
```

4. Durchschnittliche Anzahl der Bestellungen pro Kunde

```
SELECT AVG(num_orders)
FROM (
  SELECT fk_customer_id, COUNT(id) AS num_orders
  FROM order_entry
  GROUP BY fk_customer_id
) AS customer_orders;
```

24. Aufgabe 8.1: Komplexe SQL Queries

1. Adressliste der Kunden mit Adresse, Stadt und Land

```
SELECT customer.first_name, customer.last_name, address.address,  
address.district, city.city, country.country  
FROM  
customer  
LEFT JOIN address ON customer.address_id = address.address_id  
LEFT JOIN city ON address.city_id = city.city_id  
LEFT JOIN country ON city.country_id = country.country_id;
```

2. Vorname und Nachname aller Schauspieler des Films "HARRY IDAHO"

```
SELECT actor.first_name, actor.last_name, film.title  
FROM actor  
INNER JOIN film_actor  
ON actor.actor_id = film_actor.actor_id  
INNER JOIN film  
ON film.film_id = film_actor.film_id  
WHERE film.title = 'HARRY IDAHO';
```

3. Namen aller Kategorien und Anzahl Filme

```
SELECT category.name, COUNT(film.title) AS Anzahl  
FROM category  
JOIN film_category ON category.category_id = film_category.category_id  
JOIN film ON film_category.film_id = film.film_id  
GROUP BY category.name;
```

4. Namen der Kategorien die weniger als 60 Filme enthalten

```
SELECT name FROM category  
WHERE category_id IN (  
SELECT category_id  
FROM film_category  
GROUP BY category_id  
HAVING COUNT(*) < 60  
);
```

5. Länder mit mehr als 30 Städten

```
SELECT country
FROM city
JOIN country ON city.country_id = country.country_id
GROUP BY country
HAVING COUNT(*) > 30;
```

6. Durchschnittliche Länge der Filme in der Kategorie "Comedy", welche "PG-13" geratet sind

7. Umsatz des Unternehmens (Payments) mit Filmen, die als "NC-17" geratet wurden

8. Tammy Sanders aus Changhwa hat alle Filme zurückgebracht. Erfassen Sie alle notwendigen Änderungen in der Datenbank

9. Entfernen Sie alle Filme, in welchen "Dan Torn" mitspielt, aus der Kategorie Drama

10. Titel der Filme mit der längsten Spieldauer